

SCS 2312 - Compiler Assignment

A.V.A.R.L.Premathilaka - 23001496

February 14, 2026

Contents

1	Introduction	2
2	Grammar Definition	2
2.1	Lexer Grammar	2
2.2	Parser Grammar	2
3	Code Explanation	3
3.1	Lexer	3
3.1.1	The Lexer DFA	3
3.2	Parser	4
3.2.1	Limitations of the Parser	4
3.3	Executor	4
4	Usage and Testing	4

1 Introduction

Main goal for this assignment was to design a compiler that represents a real world compiler as closely as possible. The main components and workflow of the compiler is as follows:

- **Lexer** - Takes the input file and produces a stream of tokens
- **Parser** - Takes the stream of tokens and produces a parse tree
- **Executor** - Takes that parse tree and executes the program using a symbol table

2 Grammar Definition

The grammar below are defined in the syntax of ANTLR4

2.1 Lexer Grammar

```
STMT_END      :    ';' ;
KW_INT        :    'int' ;
ADD_OP        :    '+' ;
ASSIGN        :    '=' ;

ID            :    [a-zA-Z_] [a-zA-Z_0-9]* ;
INT_LITERAL   :    [0-9]+ ;

L_PAREN       :    '(' ;
R_PAREN       :    ')' ;

WS            :    [\r\n \t]+ -> skip ;
```

2.2 Parser Grammar

```
prog          :    stmt_list ;

stmt_list     :    stmt
               |    stmt_list stmt
               ;

stmt          :    funcCall
               |    declaration
               ;

declaration   :    type ID ASSIGN expr STMT_END
               ;

funcCall      :    ID L_PAREN args R_PAREN STMT_END
               |    ID L_PAREN R_PAREN STMT_END
```

```

;

args      :   term
;

term      :   literal
          |   ID
;

literal   :   INT_LITERAL
;

type      :   KW_INT
;

expr      :   term
          |   expr ADD_OP term
;

```

3 Code Explanation

3.1 Lexer

3.1.1 The Lexer DFA

Here we have implemented the lexer using an actual DFA in the file (`lexerDFA.c`). The State diagram for the DFA is as shown below:

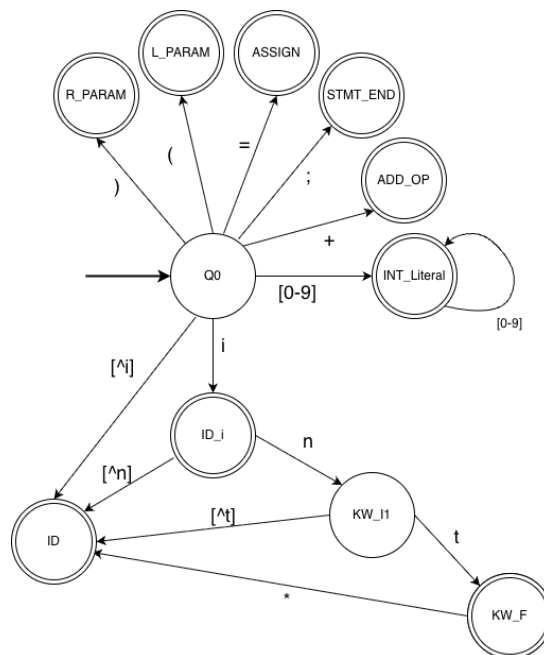


Figure 1: If a state receives an input for which there is no transition it's considered to go to a trap state

This DFA is implemented using a 2D array where rows represents the state and the columns represents the possible input character types and transitions between states can be as simple as,

```
new_state = transition_table[current_state][input_char_type];
```

This lexer implementation returns the matching token thus it can be feed into the next stage of the pipeline without storing the entire token stream in memory

3.2 Parser

With the pipeline design of the lexer a bottom up parser would be the most optimal and efficient design but due to the size of the grammar the manual creation of a the tables didn't seem feasible thus a **recursive descent parser** was implemented in the file (`parser.c`).

3.2.1 Limitations of the Parser

- **Issue:** Since this is a recursive descent parser the grammar had to be right recursive thus would make the associativity of the operators incorrect.
- **Solution:** To solve this i've implemented the recursive rules with iteration instead of recursion thus the associativity is maintained even though only the plus operator was needed for this assignment.

The parser then finally uses the convenience functions defined in the file (`parseTree.c`) to create the parse tree which is then used by the executor

3.3 Executor

The executor is implemented in the file (`executor.c`) and it uses a symbol table implemented in the file (`symbolTable.c`) to store the variables and their values. The executor traverses the parse tree and executes the program by evaluating the expressions and updating the symbol table accordingly.

4 Usage and Testing

Even though this was asked to build using just a single file due to the modular implementation and out-scope features i've implemented the compiler using multiple files.

I've attached a pre-processor compiled version of the code in a single file named (`compiler.c`) which can be used but for referencing the code please refer to the individual files in the source code folder.

To compile the code in the source code folder you can simply run the command,

```
make
```

This will create an executable named `a.out` which can be run using the command,

```
./a.out <input_file> -i -d <delay_in_ms>
```

The arguments are as follows,

- `<input_file>` - The file containing the source code to be executed
- `-i` - when this flag is used the compiler will pause at each token read and wait for the user to press enter before continuing this is useful for debugging and understanding the flow of the compiler and the creation of the parse tree
- `-d <delay_in_ms>` - This flag is used to slow down the execution of the compiler by adding a delay between each token read. Cannot be used with the `-i` flag